PRACTICAL NO. 1

Introduction to Generative AI APIs

Python application to interact with a Generative AI model API and generate responses for user-provided prompts

Student Name	_____
Class / Division	_____
Roll Number	_____
Subject	Generative Artificial Intelligence
Date	_____

Practical record / examination write-up

1. Aim

To develop a Python application that interacts with a Generative AI model API and generates responses for user-provided prompts.

2. Objectives

- To understand how a Python application communicates with a Generative AI API.
- To securely configure and use an API key.
- To accept prompts from a user through the command line.
- To send prompts to a Generative AI model and receive generated responses.
- To display the generated responses to the user.

3. Hardware Requirements

- Computer or laptop.
- Minimum 4 GB RAM recommended.
- Internet connection for communication with the hosted AI API.

4. Software Requirements

- Operating System: Windows, Linux or macOS.
- Python 3.10 or later.
- Visual Studio Code or any Python IDE.
- Python libraries: groq and python-dotenv.
- Groq API key and an available Groq model.

5. Theory

Generative Artificial Intelligence (Generative AI) refers to AI systems that can create new content such as text, images, audio, video or code based on user instructions.

An Application Programming Interface (API) allows one software application to communicate with another. A Generative AI API allows a Python application to send a prompt to an AI model and receive the model's generated response.

In this practical, a Python application uses the Groq API to communicate with a hosted language model. The user enters a prompt, the application sends it to the model, and the model returns a response. The response is then displayed on the screen.

An API key is used to authenticate the application. In this experiment, the key is stored in a .env file and loaded using python-dotenv rather than being written directly in the source code.

6. Main Components

Component	Purpose
Python	Used to write the application.
Generative AI model	Understands prompts and generates responses.
Groq API	Provides an interface to access the hosted model.
API key	Authenticates the application.
python-dotenv	Loads the API key from the .env file.
User prompt	Instruction or question entered by the user.

Component	Purpose
API response	Generated text returned by the model.

7. Project Folder Structure

```
Generative_AI_API/
|
|-- main.py
|-- .env
```

The **main.py** file contains the Python application. The **.env** file stores the API key securely outside the Python source code.

8. API Key Configuration

Create a Groq account, generate an API key from the Groq Console, and create a file named **.env** in the project folder.

```
GROQ_API_KEY=your_groq_api_key_here
```

Generate or manage the API key at https://console.groq.com/keys. Keep the key private and do not upload the .env file to a public repository.

9. Installation of Required Libraries

Open the terminal in the project folder and execute the following command:

```
pip install groq python-dotenv
```

10. Python Program (main.py)

```
import os
from dotenv import load_dotenv
from groq import Groq

# 1. Load environment variables
load_dotenv()

# 2. Read the API key
api_key = os.getenv("GROQ_API_KEY")

if not api_key:
    raise ValueError("Groq API key not found!")

# 3. Initialize the Groq client
client = Groq(api_key=api_key)

print("\\n===== GENERATIVE AI APPLICATION =====")
print("Enter a prompt to get an AI response.")
print("Type 'exit' to stop.\\n")

# 4. Accept prompts and generate responses
while True:
    user_prompt = input("You: ").strip()

    if user_prompt.lower() == "exit":
        print("AI: Goodbye!")
        break

    if not user_prompt:
        print("Please enter a prompt.\\n")
        continue

    # 5. Send prompt to the Generative AI model
    response = client.chat.completions.create(
        model="openai/gpt-oss-20b",
        messages=[
            {
                "role": "user",
                "content": user_prompt
            }
        ],
        temperature=0.7
    )

    # 6. Display generated response
    answer = response.choices[0].message.content
    print("\\nAI:", answer, "\\n")
```

11. Algorithm

- Start the program.
- Load the environment variables from the .env file.
- Read the Groq API key.
- Initialize the Groq client.
- Accept a prompt from the user.
- If the user enters exit, terminate the program.
- Send the prompt to the Generative AI model through the API.
- Receive the generated response.
- Display the response on the screen.
- Repeat the process for the next prompt.

12. Execution

Run the Python program from the terminal:

```
python main.py
```

13. Sample Output

```
===== GENERATIVE AI APPLICATION =====
Enter a prompt to get an AI response.
Type 'exit' to stop.

You: What is Generative AI?

AI: Generative AI is a type of artificial
intelligence that creates new content such
as text, images, audio and code based on
user instructions.

You: Give me three applications of AI.

AI: Three applications of AI are:
1. Healthcare diagnosis support.
2. Recommendation systems.
3. Virtual assistants.

You: exit
AI: Goodbye!
```

The output shown above is illustrative. The actual response is generated by the selected model and may vary for the same prompt.

14. Explanation of Important Code

load_dotenv() — Loads variables stored in the .env file into the program's environment.

os.getenv('GROQ_API_KEY') — Reads the API key from the environment.

Groq(api_key=api_key) — Creates a client used to communicate with the Groq API.

input() — Accepts a prompt entered by the user.

client.chat.completions.create() — Sends the prompt to the selected chat model through the API.

model — Specifies which language model should generate the response.

messages — Contains the user prompt sent to the model.

temperature — Controls the randomness of generated responses.

response.choices[0].message.content — Extracts the generated text from the API response.

while True — Keeps the application running until the user enters exit.

15. Result

The Python application was successfully developed to interact with a Generative AI model through the Groq API. It accepts user-provided prompts, sends them to the model, receives generated responses and displays those responses to the user.

16. Conclusion

This practical demonstrates how to integrate a Generative AI model API with a Python application. It provides an understanding of API authentication, prompt submission and response generation. The application can be extended to support different prompt types and user interfaces.

17. Viva Questions and Answers

1. What is Generative AI?
AI that can create new content such as text, images, audio and code.

2. What is an API?
An interface that allows software applications to communicate.

3. What is a Generative AI API?
An API that lets an application send prompts to a generative model and receive generated content.

4. What is Groq?
An AI inference platform that provides access to language models.

5. What is an API key?
A credential used to authenticate an application with an API.

6. What is a prompt?
An instruction or question given to an AI model.

7. What is an LLM?
A Large Language Model trained to understand and generate text.

8. What is python-dotenv?
A library used to load environment variables from a .env file.

9. What is temperature?
A setting that controls the randomness of model responses.

10. What does the API return?
A response containing generated content from the model.

11. Why should an API key be kept secret?
To prevent unauthorized use of the API account.

12. Does the application work without internet?
No. It needs internet to communicate with the hosted API.

18. Precautions and Limitations

- Keep the API key private. Do not hardcode it in the source code or share it publicly.
- Ensure that the computer has an active internet connection.
- Check that the API key is valid and that the selected model is available to the account.
- The API may have rate limits, usage limits or account-specific restrictions.
- Generated responses may contain mistakes; verify important information.
- The application sends each prompt as a separate request and does not maintain conversation history.
- Do not send confidential or sensitive information to an external AI API.